

MindSync AI — Exhaustive Codebase & Technical MasterBook

Complete Line-by-Line Architecture, Codefile Manual, Feature Operating Guides, & Viva Defense Handbook

System Name: MindSync AI Wellness Companion	Target Audience: IT Professionals & Software Engineers
Frontend Stack: Flutter 3.x, Dart 3.x, Riverpod, Hive	Backend Stack: FastAPI, Python 3.14, Pydantic
ML & XAI Engine: Random Forest + SHAP (Scikit-Learn)	GenAI Acceleration: Groq LPU's (Llama 3.3 70B)
Deployment Platform: Render Cloud Platform	Live Backend: https://ai-mental-wellness-companion-for-it.onrender.com/

1. System Technical Overview & Microservices Architecture

MindSync AI is a preventive digital health shield engineered specifically to reduce IT developer burnout, sedentary desk exhaustion, and workplace mental fatigue. The system integrates hardware accelerometer sensors, subjective biometric telemetry, machine learning risk classification, explainable AI, and generative language models into a cohesive, high-throughput platform.

```
+-----+ | FLUTTER MOBILE
FRONTEND LAYER | | - UI Pages: Dashboard, Mood Logging, Burnout Predictor, AI Chatbot, Reports | | - State
Management: Riverpod (authStateProvider, dashboardProviders) | | - Hardware Sensors: Pedometer (Step Stream)
& Geolocator (Weather) | | - Storage: Hive Key-Value Storage (authBox, settingsBox, moodSyncBox) |
+-----+ | HTTP / REST API
(Bearer Firebase Token) v
+-----+ | FASTAPI BACKEND
MICROSERVICES LAYER | | - Security & Auth: JWT Bearer Validation + Fallback Payload Decoder | | - Endpoints:
/health, /api/predict/burnout, /api/recommendations/*, /api/analytics | | - Firebase Admin SDK: Project
'mindsync-ai-18fbb' Sync |
+-----+ |
+-----+ v v +-----+
+-----+ | ML & XAI SUBSYSTEM | | GENERATIVE AI & LLM ORCHESTRATION | | -
Model: Random Forest Classifier | | - Model: Llama 3.3 70B Versatile | | - Features: 9 Telemetry Metrics | | -
Hardware: Groq LPU Accelerated | | - Explainability: SHAP Values | | - Safety: 988 Crisis Override |
+-----+ +-----+
```

2. Complete Codebase File Inventory & Architecture Map

File Path	Subsystem	Lines / Size	Detailed Code Function & Purpose
backend/app/main.py	Backend	65 lines	FastAPI app instance creation, CORS middleware setup, router mounting (/health, /api/predict, /api/recommendations, /api/analytics).
backend/app/core/security.py	Backend	45 lines	JWT Bearer validation. Verifies Firebase ID tokens via Firebase Admin SDK; includes _decode_jwt_payload_fallback for Render cloud execution.
backend/app/core/firebase.py	Backend	52 lines	FirebaseService singleton class. Initializes Firebase Admin SDK with explicit projectId 'mindsync-ai-18fbb' and Firestore db client.
backend/app/ml/pipeline.py	Backend ML	110 lines	BurnoutPipeline class. Loads pre-trained Random Forest model, runs inference on 9-feature telemetry vector, and computes SHAP feature importance.
backend/app/ml/recommendation_engine.py	Backend ML	300 lines	HybridRecommendationEngine class. Evaluates rule-based health checks + ML risk weights + Groq AI generative recommendations.
backend/app/services/ai_orchestrator.py	Backend Service	91 lines	AIOrchestrator class. Executes Groq Llama 3.3 70B API requests with exponential retry backoff and CacheService TTL integration.
backend/app/api/endpoints/predict.py	Backend API	66 lines	POST /api/predict/burnout endpoint. Pydantic request validation for 9 biometric parameters, returns risk category and SHAP factors.
backend/app/api/endpoints/recommendations.py	Backend API	138 lines	POST /api/recommendations/generate & /daily-summary endpoints. Formats metrics and yields personalized developer lifestyle habits.
backend/app/api/endpoints/analytics.py	Backend API	570 lines	POST /api/analytics/report/export endpoint. Generates formatted 2.9KB PDF binary stream document using ReportLab for client download.
frontend/lib/main.dart	Frontend	107 lines	Flutter app entry point. Initializes WidgetsBinding, Firebase, StorageService, loads .env, wraps Riverpod ProviderScope, mounts error guards.
frontend/lib/core/config/app_config.dart	Frontend Core	25 lines	Reads BACKEND_URL from .env (default: Render URL), API keys, responsive layout breakpoints, and demoMode toggle.
frontend/lib/core/network/dio_client.dart	Frontend Network	91 lines	Configures Dio HTTP client with base URL, 15s connect/receive timeouts, LogInterceptor, and mounts AuthInterceptor.
frontend/lib/core/network/auth_interceptor.dart	Frontend Security	23 lines	Intercepts outgoing HTTP calls, fetches stored Firebase ID token from Hive authBox, and attaches Authorization: Bearer header.
frontend/lib/core/services/pedometer_service.dart	Frontend Service	85 lines	Hardware Pedometer stepCountStream subscriber. Persists step count in Hive and falls back smoothly to realistic step simulation.
frontend/lib/features/authentication/	Frontend Auth	208 lines	FirebaseAuthDataSourceImpl & AuthNotifier. Handles sign in, sign up, password reset, email verification, and token persistence.
frontend/lib/features/chat/	Frontend Chat	238 lines	ChatRemoteDataSourceImpl. AI Chat interface. Calls Groq Llama 3.3 70B with crisis safety overrides & IT wellness fallbacks.
frontend/lib/features/reports/	Frontend Reports	163 lines	ReportHistoryPanel widget. Downloads PDF/CSV report streams and uses OpenFileX & path_provider to auto-open files on device.

3. Detailed File-by-File & Line-by-Line Code Explanations

3.1 Backend Security & JWT Fallback (security.py)

Purpose: Authenticates every incoming HTTP request. Verifies Firebase JWT tokens using Firebase Admin SDK and provides a fallback JWT payload decoder for cloud deployment on Render.

```
def _decode_jwt_payload_fallback(token: str) -> Optional[str]: # Decodes raw JWT payload
(header.payload.signature) if Firebase Admin lacks service account JSON on Render try: parts =
token.split('.') if len(parts) == 3: payload_b64 = parts[1] + '=' * (-len(parts[1]) % 4) # Add base64 padding
payload_data = json.loads(base64.b64decode(payload_b64).decode('utf-8')) return payload_data.get("user_id")
or payload_data.get("sub") or payload_data.get("uid") except Exception: pass return None
```

3.2 Random Forest ML & SHAP XAI Pipeline (pipeline.py)

Purpose: Executes supervised machine learning inference on the 9-feature telemetry vector and calculates SHAP values to explain feature contributions.

```
class BurnoutPipeline: def predict(self, features: dict) -> dict: # Construct feature DataFrame matching
exact trained feature ordering X = pd.DataFrame([features]) prob = float(self.model.predict_proba(X)[0][1]) *
100.0 # Calculate Risk % risk_level = "High" if prob >= 70.0 else ("Medium" if prob >= 40.0 else "Low") #
Calculate SHAP Shapley values for Explainable AI (XAI) explainer = shap.TreeExplainer(self.model) shap_values
= explainer.shap_values(X) important_factors = self._extract_top_factors(shap_values, X) return
{"burnoutRisk": risk_level, "riskScore": prob, "importantFactors": important_factors}
```

3.3 Frontend Auth Interceptor (auth_interceptor.dart)

Purpose: Intercepts all outgoing HTTP requests from the mobile app and attaches the stored Firebase ID token as a Bearer authorization header.

```
class AuthInterceptor extends Interceptor { @override void onRequest(RequestOptions options,
RequestInterceptorHandler handler) { if (Hive.isBoxOpen(AppConstants.authBoxName)) { final authBox =
Hive.box(AppConstants.authBoxName); final token = authBox.get(AppConstants.keyJwtToken) as String?; if (token
!= null && token.isNotEmpty) { options.headers['Authorization'] = 'Bearer $token'; // Attach Firebase Bearer
token } } super.onRequest(options, handler); } }
```

4. Exhaustive Operating Guides for All Application Features

• Feature 1: User Authentication & Session Persistence:

Users register or log in using email and password. Upon sign-in, FirebaseAuthDataSourceImpl retrieves the user's Firebase ID token and stores it securely in Hive's authBox (key: 'jwt_token'). If the app is closed and reopened, getCurrentUser() automatically restores the token and verifies email verification status. Auto-signout triggers after 15 minutes of inactivity.

• Feature 2: Dashboard & Real-Time Step Counter:

The Dashboard displays the daily wellness score, steps progress bar, weather card, and motivational quote. Hardware step counting is managed by pedometer_service.dart. It listens to Pedometer.stepCountStream and saves steps in Hive. If run in an emulator without hardware sensors, it smoothly generates realistic step updates so tests pass cleanly.

• Feature 3: Daily Mood & Biometric Check-In:

Users tap 'Log Mood' to enter their daily metrics: Mood Rating (1-10 scale), Sleep Duration (hours), Stress Level (1-10), Water Intake (ml/glasses), and Active Exercise Minutes. Submitting syncs data to Firebase Firestore and triggers an automatic refresh of analytics trends.

• Feature 4: Supervised ML Burnout Risk Forecast & SHAP XAI:

Users access the Burnout Predictor screen to calculate their burnout index. The app sends a 9-parameter vector to POST /api/predict/burnout on Render. The backend Random Forest model outputs the burnout probability % and displays top SHAP attribution factors (e.g. '+35% risk from 13h work day').

• Feature 5: AI Chatbot & Crisis Safety Overrides:

Users open the AI Chat tab to interact with MindSync AI. The chatbot calls Llama 3.3 70B via Groq LPUs. If crisis keywords like 'suicide', 'harm myself', or 'want to die' are detected, the app immediately intercepts the prompt locally and returns 988 Suicide & Crisis Lifeline contact resources.

• Feature 6: Analytics & Automatic PDF/CSV Report Opening:

Users select time ranges (7 Days, 30 Days, Custom) on the Reports page. Tapping 'Export PDF' sends a request to POST /api/analytics/report/export. The backend generates a formatted 2.9KB PDF binary stream. Upon download, the app uses path_provider to save the file and OpenFilex.open(file.path) to automatically open the PDF viewer on the device.

5. Complete Mathematical Formulations & ML Proofs

1. Entropy (Information Theory):

$$H(S) = - \sum [p_i * \log_2(p_i)]$$

Quantifies impurity in dataset S across target risk categories (Low, Medium, High).

2. Information Gain:

$$IG(S, A) = H(S) - \sum [(|S_v| / |S|) * H(S_v)]$$

Measures reduction in entropy when splitting decision trees on biometric feature A (e.g. sleep hours).

3. Gini Impurity:

$$Gini(S) = 1 - \sum (p_i)^2$$

Used by Scikit-Learn Random Forest Classifier as node splitting metric to minimize sub-tree variance.

4. SHAP Shapley Values (Game Theory):

$$\phi_i(v) = \sum [(|S|! * (|N| - |S| - 1)! / |N|!) * [v(S \cup \{i\}) - v(S)]$$

Calculates exact marginal feature attribution for every individual prediction.

6. Viva Presentation Defense MasterScript & Examiner Q&A;

5-Step Presentation Storyline for Viva Examination:

- **Step 1: The Problem:** Developer burnout is at an all-time high due to tight sprint deadlines, sedentary desk hours, code review stress, and context switching. Generic apps only offer manual journaling without predictive technology.
- **Step 2: The Solution:** MindSync AI acts as a preventive digital health shield. It combines hardware step tracking, subjective mood logs, supervised ML prediction, explainable AI, and generative LLMs.
- **Step 3: The Architecture:** Flutter frontend communicating asynchronously via Firebase JWT Bearer tokens to a FastAPI microservices backend deployed on Render, powered by Random Forest and Groq-accelerated Llama 3.3.
- **Step 4: Key Technical Highlights:** SHAP XAI for transparent risk attribution, hardware pedometer stream fallback, automated PDF report generation and auto-opening via OpenFilex, and local 988 crisis safety overrides.
- **Step 5: Enterprise Impact:** Scalable B2B SaaS model targeting IT organizations to reduce developer turnover, lower absenteeism, and improve software engineering productivity.

Top 10 Technical Viva Questions & Examiner Defense Answers:

Q1: How does authentication work between Flutter and FastAPI on Render?

Answer: Flutter uses Firebase Auth for user login. Upon login, the ID token is persisted in Hive (authBox). Every outgoing Dio HTTP request attaches this token as a Bearer header. FastAPI verifies the token via Firebase Admin SDK, with a JWT payload fallback decoder if service account keys are absent on Render.

Q2: Why did you choose Random Forest instead of Deep Learning (Neural Networks)?

Answer: Random Forest provides superior classification accuracy on tabular biometric data without overfitting. More importantly, tree ensembles integrate directly with SHAP for exact Explainable AI (XAI) attribution, whereas neural networks act as opaque black boxes.

Q3: What is the purpose of SHAP values in your system?

Answer: SHAP values provide game-theoretic mathematical proof for why a user received a specific burnout risk score. It explains whether high work hours (+35%) or poor sleep (+25%) contributed more to their stress, providing actionable transparency.

Q4: How does the step counter work automatically?

Answer: The app uses pedometer_service.dart which listens to hardware accelerometer sensors via Pedometer.stepCountStream. If hardware sensors are absent (e.g. emulators), it smoothly falls back to a realistic step simulation engine so the UI never crashes.

Q5: How do you handle API key failures or offline mode?

Answer: The app features a multi-tiered fallback architecture. If Groq API keys are unconfigured or network requests fail, the app identifies placeholder patterns ('your_', 'mock_') and seamlessly falls back to local intelligent IT wellness rules and Hive cached data.

Q6: How does report auto-opening work after export?

Answer: Upon downloading a PDF or CSV report, the app saves the file to local device storage using path_provider and immediately invokes OpenFilex.open(file.path) to launch the native PDF reader automatically.

Q7: How do you enforce crisis safety for self-harm keywords?

Answer: Before sending prompts to Groq LLM, chat_remote_datasource.dart checks for crisis keywords ('suicide', 'kill myself', 'want to die'). If detected, it immediately overrides with 988 Suicide & Crisis Lifeline resources without invoking external APIs.

Q8: How is state managed across Flutter screens?

Answer: We use Riverpod (authStateProvider, dashboardProviders, analyticsSummaryStateProvider). It provides reactive, immutable state management with dependency injection and easy unit testing.

Q9: What is the role of Hive storage in your architecture?

Answer: Hive is a lightweight, fast key-value database written in pure Dart. It caches authentication tokens, user settings, offline mood logs, and dashboard summaries for instant app startups and offline support.

Q10: What is the deployment architecture on Render?

Answer: FastAPI backend is containerized and hosted on Render. Render watches our main Git branch and automatically redeploys microservices whenever changes are pushed to GitHub.